Session 2.5

Solving the Growth Model as Supervised Learning on the Euler Equation

The network is the policy, sampled states are the inputs, and the Euler equation supplies the labels



© Zhigang Feng Redistribution prohibited without permission

Zhigang Feng

Session 2.5

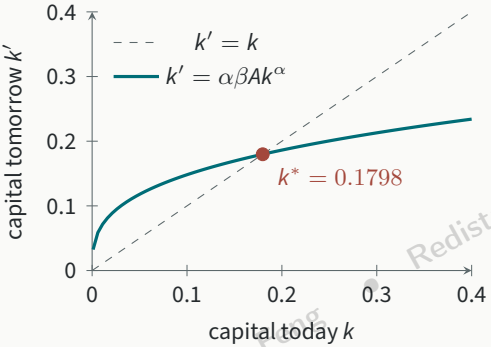
The Model

One unknown function, and the condition it must satisfy



© Zhigang Feng Redistribution prohibited without permission

The Growth Model



© Zhigang Feng
Redistribution prohibited without permission

- Maximise $\sum_t \beta^t u(c_t)$ subject to $c + k' = Ak^\alpha + (1-\delta)k$.
- The answer is a **policy function** $k' = \pi(k)$: capital tomorrow, given capital today.
- Rung 0: log utility, $\delta = 1$, $\beta = 0.96$. The policy is known: $k' = \alpha\beta Ak^\alpha$ (left).
- Rung 1: $u = c^{1-\gamma}/(1-\gamma)$, $\gamma = 2$, $\delta = 0.025$, $\beta = 0.99$. No closed form; $k^* = 28.35$.
- **Solving the model means finding the function π.**

Source: Both rungs: $\alpha = 0.33$, $A = 1$; the lab's names. Brock & Mirman, *JET* 4(3), 1972; the model of Session 1.4.

The Euler Equation, and Its Unit-Free Residual

- The **Euler equation**: at an optimum, giving up one unit of consumption today and eating the proceeds tomorrow must leave utility unchanged,

$$u'(c) = \beta\, u'(c')\, R', \qquad R' = \alpha A k'^{\alpha-1} + 1 - \delta.$$

- Under a candidate policy π, both sides can be computed at any k: $k' = \pi(k)$, c from the budget, then $k'' = \pi(k')$ and c'. Their gap is the **residual**.
- Divide and take logs, so the gap has no units:

$$r_\pi(k) = \log \frac{\beta\, u'(c')R'}{u'(c)}.$$

- $r = 0.01$ reads as a one per cent wedge between the two sides, at any calibration.
- **The true policy makes the residual zero at every k.**

© Zhigang Feng · Redistribution prohibited without permission

Session 2.5

How We Usually Solve It

Time iteration on the Euler equation



© Zhigang Feng
Redistribution prohibited without permission

Time Iteration: the Algorithm

- Grid $k_1 < \cdots < k_N$; resources $f(k) = Ak^\alpha + (1-\delta)k$; a first guess π_0.
- Round n: at each node k_i, solve one equation for today's choice $x = k'$, with tomorrow's behaviour given by last round's policy π_n:

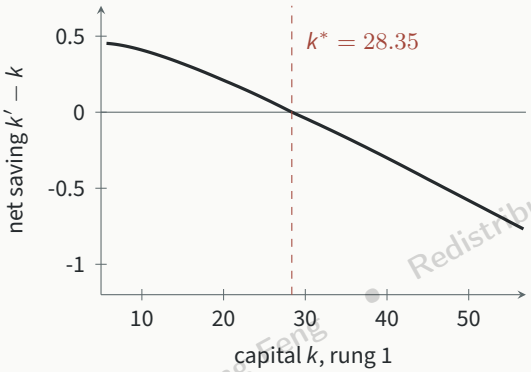
$$u'\big(f(k_i) - x\big) = \beta\, u'\big(f(x) - \pi_n(x)\big)\,\big(\alpha A x^{\alpha-1} + 1 - \delta\big).$$

- Set $\pi_{n+1}(k_i) = x_i$. Between nodes, interpolate.
- Stop when $\max_i |\pi_{n+1}(k_i) - \pi_n(k_i)| / \pi_n(k_i) < 10^{-10}$.
- **Each round solves the Euler equation exactly at the nodes, given the previous round's policy.**

© Zhigang Feng. Redistribution prohibited without permission

Source: W. Coleman, "Equilibrium in a production economy with an income tax", *Econometrica* 59(4), 1991. Lab: 400 nodes, log-spaced; each root by bisection.

Time Iteration's Answer, and Its Cost



- Below k^* the economy saves, above it dissaves: capital moves to the steady state.
- 400 nodes, 292 rounds, 1.5 seconds on one CPU core.
- $\log_{10}$ max Euler error -3.44: one dollar of consumption in about 2,750.
- With d state variables, N nodes per variable become N^d nodes.
- **Fast and accurate in one dimension. This is the answer the network must match.**

Source: Session 2 lab notebook, block D (rung 1).

© Zhigang Feng
Redistribution prohibited without permission

Session 2.5

Reframing It as Supervised Learning

The network is the policy; the Euler equation supplies the labels



© Zhigang Feng
Redistribution prohibited without permission

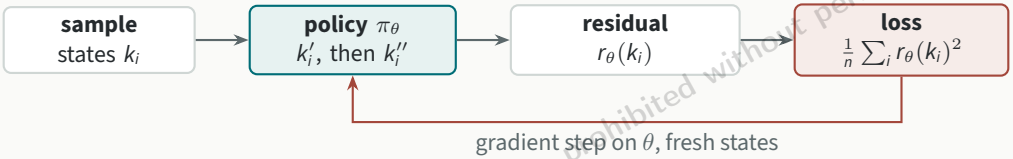
Solving a Model as Supervised Learning: the Dictionary

Supervised learning	Digits (Decks 2.1–2.3)	Growth model (this deck)
inputs x_i	images	sampled capital stocks k_i
model f_θ	classifier	the policy $k' = \pi_\theta(k)$
prediction error	$\hat{p}$ against the label	the Euler residual $r_\theta(k_i)$
label	the true digit	**zero**, at every state

© Zhirang Feng · Redistribution prohibited without permission

- Time iteration hands the Euler equation a grid. Here a network is the policy, and the equation becomes the loss.
- Nobody hands us the right k' at each k. The optimality condition says what a right answer must satisfy.
- **Inputs are free: draw as many states as you like, fresh at every step.**

From Residual to Loss: Zero Is the Label

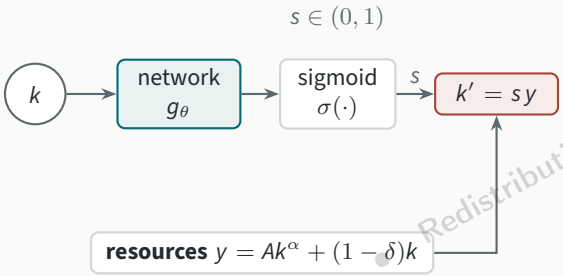


© Zhiyang Feng · Redistribution prohibited without permission

- Squared error against a label of zero: $\ell_i = (r_\theta(k_i) - 0)^2$. The training loop is Deck 2.4's five lines.
- The policy is evaluated twice, at k and at k', so gradients flow through both.
- Network: two tanh hidden layers of 32 units; 256 fresh states per step; Adam, 500 steps on rung 0, 3,000 on rung 1.
- **No benchmark policy ever enters the loss. Benchmarks are kept for testing.**

Source: Session 2 lab notebook, block D. Azinovic, Gaegauf & Scheidegger, "Deep equilibrium nets", *IER* 63(4), 2022.

Feasibility by Construction: the Sigmoid Share



© Zhigang Feng Redistribution prohibited without permission

- **Feasibility**: $c > 0$ and $k' > 0$. One infeasible state makes $\log c$ undefined and ends training.
- The network outputs a *saving share* $s = \sigma(g_\theta) \in (0, 1)$, so $k' = s\,y$ and $c = (1 - s)\,y$ are positive at every θ.
- The input is $\log k$, rescaled to $[-1, 1]$.
- **A constraint written into the architecture cannot be violated, and costs no penalty term.**

The Code: Policy and Residual

```
def saving_share(self, k):
    x = (torch.log(k) - self.log_center) / self.log_scale
    return torch.sigmoid(self.net(x))

def forward(self, k):
    return self.resources(k) * self.saving_share(k)

def log_residual(model, k):
    c, rhs = euler_rhs(model, k)
    return torch.log(rhs) + model.rung.gamma * torch.log(c)

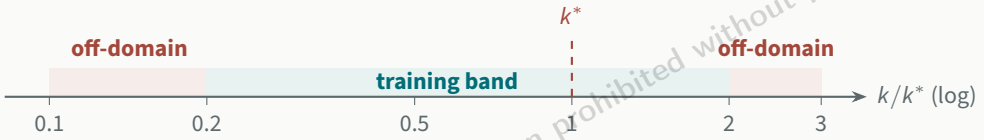
loss = log_residual(model, k).square().mean()
```

- euler_rhs returns c and $\beta u'(c')R'$; with $u'(c) = c^{-\gamma}$, line 10 is $\log \text{RHS} - \log u'(c)$.
- **Feasibility is line 6; the economics is line 10.**

Source: Session 2 lab notebook, block D, shown without the productivity shock.

© Zhenyang Peng Redistribution prohibited without permission

Where to Sample



Redistribution prohibited without permission
© Zixiang Feng

- **Training** states: uniform in $\log k$ on $[0.2,\ 2] \times k^*$, fresh at every step.
- **Held-out** states: new draws from the same band, from a generator training never used. They test coverage.
- **Off-domain** states: $[0.1, 0.2) \cup (2, 3] \times k^*$, never asked of the network. They test extrapolation.
- **The solution is claimed only where it was trained and tested.**

Session 2.5

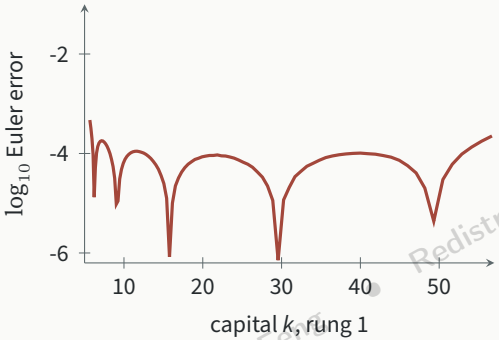
How We Test It

Residuals where it was not trained, the known answer, and the steady state



© Zhigang Feng Redistribution prohibited without permission

Held-Out and Off-Domain Residuals, on Judd's Scale



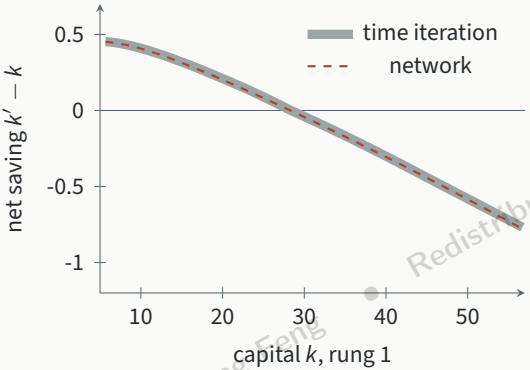
$\log_{10}$ max error	rung 0	rung 1
held-out	-1.95	-3.34
off-domain	-1.29	-1.59

- **Judd's scale**: $\log_{10}$ of the consumption error implied by the residual. -3 is one dollar in a thousand.
- Off-domain, accuracy falls by more than half a decade on rung 0 and almost two on rung 1.
- **Report the maximum, and report it where the network was not trained.**

© Zhigang Feng Redistribution prohibited without permission

Source: Session 2 lab notebook, block D. Judd, *JET* 58(2), 1992.

The Network Against the Known Answer



© Zhigang Feng
Redistribution prohibited without permission

- Benchmarks: the closed form (rung 0), time iteration (rung 1). Neither entered training.
- Largest policy error on held-out states: 0.64% on rung 0, 0.04% on rung 1.
- Off-domain: 3.4% and 1.3%.
- On rung 1 the saving share starts at its steady-state value; the lab shows a default start.
- **The two curves lie on top of each other.**

Source: Session 2 lab notebook, block D.

The Steady-State Check, and the Gates

Gate	rung 0	rung 1
feasible at every tested state	yes	yes
max policy error, held-out (gate: below 1%)	0.64%	0.04%
max policy error, off-domain (reported)	3.4%	1.3%
simulated against analytical k^* (gate: 1%)	0.47%	0.18%

© Zhitang Fang · Reproduction prohibited without permission

- Steady-state check: from $0.5\,k^*$, apply the learned policy 3,000 times. A *global* test: small errors that compound along the path show up here.
- A **gate** is a pass mark fixed before the run.
- **Both rungs pass every gate.**

Source: Session 2 lab notebook, block D; policy errors against the closed form (rung 0) and time iteration (rung 1).

Time Iteration and the Network, Side by Side

Rung 1	time iteration	network
numbers stored	400 grid values	1,153 parameters
seconds, one CPU core	1.5 (292 iterations)	4.6 (3,000 steps)
$\log_{10}$ max Euler error, held-out band	-3.44	-3.34

Redistribution prohibited without permission

- In one dimension the classical method ties on accuracy and wins on the rest.
- A grid costs N^d; the network's cost grows with the sample. Its case is dimension.
- **Learn the method where it loses, so that you can trust it where nothing else runs.**
- Next, Deck 2.6: the policy was a plain dense network. What if the architecture itself carried the economics?

Source: Session 2 lab notebook, block D (rung 1: CRRA $\gamma = 2, \delta = 0.025, \alpha = 0.33, \beta = 0.99$).